

EFF Membership Management System

Enterprise System Architecture & Solution Architecture Document

Version: 2.0 **Date:** 16 February 2026 **Classification:** Confidential **Target Scale:** 1,000,000+ Members | Enterprise Grade **Prepared For:** EFF National Command Team

Table of Contents

- 1. [Executive Summary](#)
 - 2. [System Architecture Overview](#)
 - 3. [Solution Architecture](#)
 - 4. [Hardware Requirements](#)
 - 5. [Software Requirements](#)
 - 6. [Security Requirements](#)
 - 7. [Kubernetes Deployment Strategy](#)
 - 8. [Server Procurement Recommendations](#)
 - 9. [Virtualization Strategy](#)
 - 10. [Disaster Recovery & Business Continuity](#)
 - 11. [Cost Estimates](#)
 - 12. [Implementation Roadmap](#)
-

1. Executive Summary

The EFF Membership Management System is a full-stack enterprise application designed to manage the complete lifecycle of political party membership across all 9 South African provinces. The system currently operates on a single Linode VPS and must be scaled to support **1,000,000+ active members**, **10,000+ concurrent users**, and real-time **WhatsApp bot interactions** serving millions of messages per month.

Current State

Component	Current	Target
Members	~50,000	1,000,000+
Concurrent Users	~100	10,000+
WhatsApp Messages/Month	~5,000	500,000+
SMS Campaigns/Month	~10	100+
Infrastructure	Single VPS (Linode)	Kubernetes Cluster
Database	Single PostgreSQL 16	HA Cluster (Primary + 2 Replicas)
Cache	Single Redis	Redis Sentinel (3 nodes)
Uptime SLA	Best-effort	99.95%

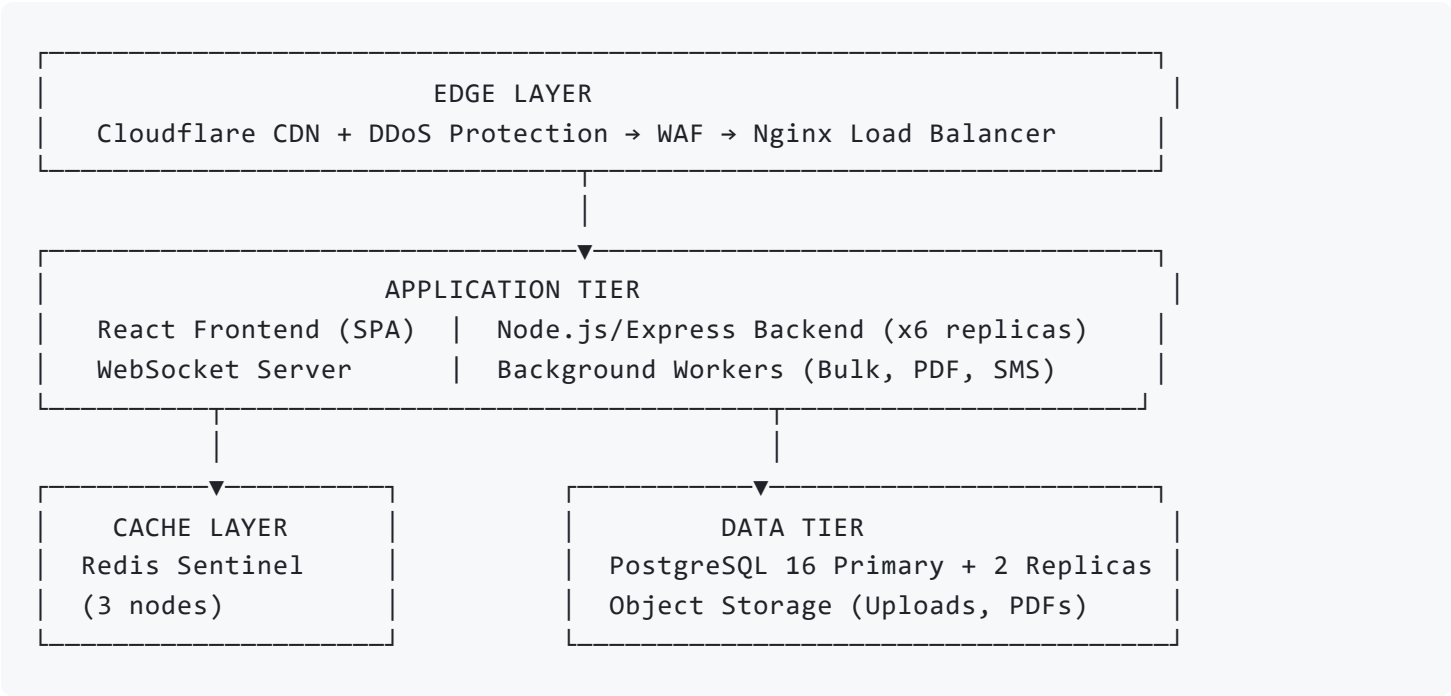
Key System Capabilities

- **Member Management:** Full CRUD, search, audit, bulk upload (Excel/CSV), digital membership cards (PDF)
- **Geographic Hierarchy:** 9 Provinces → 52 Districts → 257 Municipalities → 4,468 Wards → 23,148 Voting Districts
- **WhatsApp Bot:** Interactive bot via Meta Cloud API — member lookup, voting info, help menus, phone linking
- **SMS Campaigns:** Multi-provider SMS with birthday automation, campaign management, delivery tracking
- **Meetings & Elections:** Hierarchical meeting management, attendance registers, election tracking
- **Leadership:** Position management, leadership assignments across geographic hierarchy
- **Financial:** Payment tracking, membership renewals, bulk renewal uploads
- **Analytics:** Real-time dashboards, population pyramids, geographic analytics, Excel reports
- **Security:** JWT authentication, RBAC (4 admin levels), MFA (Email/SMS/WhatsApp OTP), audit logging
- **IEC Integration:** Electoral Commission voter verification API integration

2. System Architecture Overview

2.1 High-Level Architecture

The system follows a **multi-tier architecture** with clear separation of concerns:



See Diagram: "High-Level System Architecture" (rendered above)

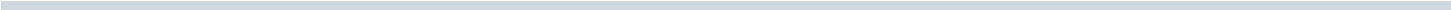
2.2 Architecture Principles

Principle	Implementation
Horizontal Scalability	Stateless backend pods behind load balancer; session state in Redis
High Availability	No single point of failure; database replication; Redis Sentinel
Security by Design	TLS 1.3, JWT + RBAC, MFA, audit logging, encrypted backups
Event-Driven	WebSocket for real-time updates; queue-based bulk processing
Geographic Redundancy	Primary in Johannesburg DC; DR in Cape Town DC
Observability	Prometheus metrics, Grafana dashboards, centralized logging

2.3 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React + Vite	React 18 / Vite 5	Single Page Application
UI Framework	Material UI (MUI)	v5	Component library

Layer	Technology	Version	Purpose
State Mgmt	Redux Toolkit	Latest	Global state
Backend	Node.js + Express	Node 20 LTS / Express 4	REST API server
Language	TypeScript	5.x	Type safety
ORM	Prisma + Raw SQL	Prisma 5	Database access (hybrid)
Database	PostgreSQL	16	Primary data store
Cache	Redis	7	Sessions, API cache
Process Mgr	PM2 / Kubernetes	Latest	Process orchestration
Reverse Proxy	Nginx	Latest	Load balancing, SSL
PDF Engine	Puppeteer + Chromium	Latest	Digital card generation
WhatsApp	Meta Cloud API	v21	Bot messaging
SMS	Multi-provider	Various	SMS campaigns
Email	SMTP (Nodemailer)	Latest	Transactional email
Containerization	Docker	24+	Application packaging
Orchestration	Kubernetes	1.29+	Container orchestration



3. Solution Architecture

3.1 Component Architecture

See Diagram: "Solution Architecture (Component View)" (rendered above)

The solution is composed of the following layers:

3.1.1 Frontend Layer (React SPA)

Module	Description	Key Pages
Auth	Login, MFA verification, OTP (Email/SMS/WhatsApp)	LoginPage, MFAVerificationPage
Dashboard	Provincial/Municipal/Ward dashboards with analytics	DashboardPage, AnalyticsPage
Members	Member CRUD, search, detail view, bulk upload	MembersPage, MemberDetailPage
Meetings	Hierarchical meeting management, attendance	MeetingsPage, AttendanceRegister
Elections	Election tracking, delegate management	ElectionsPage, DelegatesPage
Communication	SMS campaigns, templates, birthday automation	CommunicationPage, SMSCampaigns
Bulk Upload	Excel/CSV upload with real-time WebSocket progress	BulkUploadPage
Reports	Excel export, attendance registers, analytics reports	ReportsPage
Financial	Payment tracking, renewal management	FinancialPage
Admin	User management, system settings, cache management	AdminPage, SuperAdminPage
Leadership	Leadership positions and assignments	LeadershipPage

3.1.2 Backend API Layer (50+ Route Modules)

Middleware Pipeline:

Request → Rate Limiter → Security Headers → CORS → Auth (JWT) → RBAC → Cache Check → Audi

Key Route Groups:

Group	Routes	Description
Core	members, memberships, membership-applications	Member lifecycle management
Geographic	geographic, lookups, statistics	Province/District/Municipality/Ward/VD
Communication	sms, communication, birthday-sms, whatsapp	All messaging channels
Meetings	meetings, hierarchical-meetings, meeting-documents	Meeting management
Elections	elections, leadership, delegates-management	Political structures
Admin	admin-management, super-admin, system, security	System administration
Financial	financial, renewals, payment	Financial operations
Reports	analytics, views, import-export	Data analysis and export
Bulk Ops	bulk-operations, bulk-upload, file-processing	Mass data operations
Cards	digital-cards, optimized-cards	PDF membership cards
Auth	auth, mfa, emergency-access, session	Authentication and MFA
Audit	audit-logs, member-audit, ward-audit	Compliance and audit
IEC	voter-verifications, iec-api	Electoral commission

3.1.3 Service Layer (50+ Services)

Service Category	Services	Purpose
WhatsApp	whatsappBotService, whatsappMemberService, whatsappProviderMgmt	Bot logic, member ops, provider mgmt
SMS	smsManagementService, smsProviderMonitoring, deliveryTracking	Multi-provider SMS with failover
Bulk Upload	bulkUploadOrchestrator, processingQueue, validationService	Queue-based Excel/CSV processing
PDF	pdfGenerationService, optimizedDigitalCardService	Puppeteer-based card generation

Service Category	Services	Purpose
Cache	cacheService, cacheInvalidationService	Redis cache management
Analytics	analyticsService, chartGenerationService, excelReportService	Dashboard data and exports
Financial	comprehensiveFinancialService, financialTransactionQuery	Payment and renewal tracking
Email	emailService, attendanceRegisterEmailService	Transactional and report emails
Background	birthdayScheduler, viewsService, healthCheckService	Scheduled tasks
Real-time	websocketService, fileWatcherService	Live updates

3.1.4 Background Jobs

Job	Schedule	Purpose
birthdayMessageJob	Daily 6:00 AM SAST	Send birthday SMS/WhatsApp to members
meetingStatusJob	Every 30 minutes	Auto-update meeting statuses
membershipStatusJob	Daily 1:00 AM SAST	Expire memberships, update statuses
refreshMaterializedViews	Every 15 minutes	Refresh materialized views for search
DB Backup	Daily 2:00 AM SAST	Full PostgreSQL backup with encryption

3.2 Data Flow Architecture

3.2.1 Member Registration Flow

User → React Form → POST /api/v1/members → Auth MW → Validation →
→ Member Service → PostgreSQL INSERT → Cache Invalidation →
→ Audit Log → SMS/Email Notification → Response

3.2.2 WhatsApp Bot Flow

WhatsApp User → Meta Cloud API → Webhook POST /api/v1/whatsapp/webhook →
→ WhatsApp Bot Service → Session Lookup (Redis) → Intent Detection →
→ Process Message (DB Lookup if needed) → Build Response →
→ Meta API Send (Interactive Buttons/Lists) → Log Message

3.2.3 Bulk Upload Flow

Admin → Upload Excel → POST /api/v1/bulk-upload → File Storage →
→ Processing Queue (Redis) → Worker picks up job →
→ Parse Excel → Validate rows → Batch INSERT/UPSERT →
→ WebSocket progress updates (real-time) →
→ Generate Report Excel → Notify admin

3.3 Database Architecture

See Diagram: "Database Architecture" (rendered above)

3.3.1 Schema Overview (50+ Tables)

Core Tables:

Table	Est. Rows (1M scale)	Key Indexes
members_consolidated	1,000,000+	PK, id_number, cell_number, ward_code
membership_applications	200,000+	PK, applicant_id_number, status
users	5,000+	PK, email, username
audit_logs	10,000,000+	PK, entity_type, created_at, user_id

Geographic Tables:

Table	Rows	Description
provinces	9	9 South African provinces
districts	52	District municipalities
municipalities	257	Local municipalities
wards	4,468	Electoral wards
voting_districts	23,148	IEC voting districts

Communication Tables: sms_campaigns, sms_templates, sms_tracking, whatsapp_bot_sessions, whatsapp_bot_messages

Materialized Views (Auto-Refreshed):

- member_search_consolidated — Denormalized for fast full-text search
- membership_expiration_summary — Expiring/expired membership counts
- analytics_dashboard_summary — Pre-computed dashboard metrics
- birthday_members_today — Today's birthday members for SMS

4. Hardware Requirements

4.1 Deployment Scale Matrix

Small Scale (1,000–5,000 members | 100–500 concurrent users)

Component	Spec	Quantity	Monthly Cost (Est.)
App Server	4 vCPU, 8GB RAM, 100GB SSD	1	R3,500
DB Server	4 vCPU, 16GB RAM, 200GB NVMe	1	R5,000
Redis	2 vCPU, 4GB RAM, 20GB SSD	1 (co-located)	Included
Backup Storage	500GB Block Storage	1	R500
Total		3 instances	~R9,000/mo

Medium Scale (5,000–20,000 members | 500–2,000 concurrent users)

Component	Spec	Quantity	Monthly Cost (Est.)
App Server	8 vCPU, 16GB RAM, 200GB SSD	2 (load balanced)	R14,000
DB Primary	8 vCPU, 32GB RAM, 500GB NVMe	1	R12,000
DB Replica	4 vCPU, 16GB RAM, 500GB NVMe	1	R7,000
Redis	4 vCPU, 8GB RAM, 50GB SSD	1	R4,000
Load Balancer	Nginx (dedicated)	1	R2,000
Backup Storage	1TB Block Storage	1	R1,000
Total		7 instances	~R40,000/mo

Large Scale (20,000–50,000 members | 2,000–5,000 concurrent users)

Component	Spec	Quantity	Monthly Cost (Est.)
App Server	8 vCPU, 32GB RAM, 200GB SSD	4 (load balanced)	R48,000
Worker Node	8 vCPU, 16GB RAM, 100GB SSD	2	R14,000
DB Primary	16 vCPU, 64GB RAM, 1TB NVMe	1	R25,000
DB Replica	8 vCPU, 32GB RAM, 1TB NVMe	2	R24,000
Redis Sentinel	4 vCPU, 16GB RAM, 50GB SSD	3	R18,000
Load Balancer	Nginx HA (active/passive)	2	R6,000

Component	Spec	Quantity	Monthly Cost (Est.)
Monitoring	4 vCPU, 8GB RAM, 200GB SSD	1	R4,000
Backup Storage	2TB Block Storage	1	R2,000
Total		16 instances	~R141,000/mo

Enterprise Scale (1,000,000+ members | 10,000+ concurrent users) — RECOMMENDED

Component	Spec	Quantity	Monthly Cost (Est.)
K8s Master Nodes	8 vCPU, 16GB RAM, 100GB SSD	3	R24,000
K8s Worker Nodes	16 vCPU, 64GB RAM, 200GB NVMe	6	R180,000
DB Primary	32 vCPU, 128GB RAM, 2TB NVMe RAID-10	1	R65,000
DB Replica	16 vCPU, 64GB RAM, 2TB NVMe	2	R60,000
Redis Sentinel	8 vCPU, 32GB RAM, 100GB SSD	3	R36,000
Load Balancer	Dedicated L7 (Nginx Plus / HAProxy)	2	R12,000
Monitoring Stack	8 vCPU, 32GB RAM, 500GB SSD	2	R24,000
Bastion Host	2 vCPU, 4GB RAM, 50GB SSD	1	R2,000
Backup Storage	5TB Encrypted Block Storage	1	R5,000
Object Storage	1TB S3-compatible	1	R1,500
DR Site (Cape Town)	Mirror of critical infra (50%)	1 set	R150,000
Total		22+ instances	~R559,500/mo

4.2 PostgreSQL Tuning for 1M+ Members

```
# postgresql.conf – Enterprise Configuration
max_connections = 500
shared_buffers = 32GB          # 25% of 128GB RAM
effective_cache_size = 96GB    # 75% of 128GB RAM
maintenance_work_mem = 2GB
checkpoint_completion_target = 0.9
wal_buffers = 64MB
default_statistics_target = 500
random_page_cost = 1.1         # SSD optimization
effective_io_concurrency = 200  # SSD optimization
work_mem = 64MB                # Per-query memory
min_wal_size = 2GB
max_wal_size = 8GB
max_worker_processes = 16
max_parallel_workers_per_gather = 8
max_parallel_workers = 16
max_parallel_maintenance_workers = 4
huge_pages = try
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
hot_standby = on
synchronous_commit = on
```

4.3 Redis Configuration for Enterprise

```
# redis.conf – Enterprise
maxmemory 24gb
maxmemory-policy allkeys-lru
tcp-keepalive 300
timeout 0
save 900 1
save 300 10
save 60 10000
appendonly yes
appendfsync everysec
replica-lazy-flush yes
lazyfree-lazy-eviction yes
```

5. Software Requirements

5.1 Operating System

Component	OS Recommendation	Justification
K8s Nodes	Ubuntu 22.04 LTS Server	Long-term support until 2027, excellent K8s compatibility
DB Servers	Ubuntu 22.04 LTS Server	PostgreSQL optimized packages available
Bastion	Ubuntu 22.04 LTS Minimal	Reduced attack surface

5.2 Software Stack Versions

Software	Version	Purpose	Notes
Node.js	20 LTS (Hydrogen)	Backend runtime	LTS support until April 2026
TypeScript	5.4+	Type safety	Strict mode enabled
Express.js	4.18+	HTTP framework	
React	18.2+	Frontend framework	
Vite	5.x	Frontend build tool	HMR, fast builds
Material UI	5.x	UI component library	
PostgreSQL	16.x	Primary database	Logical replication, pg_stat_statements
Redis	7.2+	Cache and sessions	Sentinel for HA
Prisma	5.x	ORM (hybrid with raw SQL)	
Nginx	1.25+	Reverse proxy / LB	Or Nginx Plus for advanced features
Docker	24+	Containerization	Multi-stage builds
Kubernetes	1.29+	Orchestration	
Helm	3.14+	K8s package management	

Software	Version	Purpose	Notes
PM2	5.x	Process manager (non-K8s)	Cluster mode
Puppeteer	Latest	PDF generation	Chromium headless
Let's Encrypt	N/A	SSL certificates	Auto-renewal via cert-manager

5.3 Monitoring and Logging

Tool	Version	Purpose
Prometheus	2.50+	Metrics collection
Grafana	10.x	Dashboards and visualization
Loki	2.9+	Log aggregation (lightweight ELK alternative)
Promtail	2.9+	Log shipping agent
AlertManager	0.27+	Alert routing (PagerDuty, Slack, Email)
node-exporter	Latest	OS-level metrics
postgres_exporter	Latest	PostgreSQL metrics
redis_exporter	Latest	Redis metrics

5.4 Key Monitoring Metrics

Metric Category	Specific Metrics	Alert Threshold
API	Request rate, latency p50/p95/p99, error rate	p99 > 2s, error > 1%
Database	Connection pool, query time, replication lag	Lag > 30s, connections > 80%
Redis	Hit rate, memory usage, connected clients	Memory > 80%, hit rate < 90%
WhatsApp	Messages sent/received, webhook latency	Webhook > 5s
Workers	Queue depth, processing time, failure rate	Queue > 1000, failures > 5%
System	CPU, RAM, Disk I/O, Network	CPU > 80%, RAM > 85%, Disk > 90%

6. Security Requirements

6.1 Hardware Security

Requirement	Implementation	Priority
Physical Access	Data center with 24/7 security, biometric access, CCTV	Critical
Network Firewall	Palo Alto PA-400 or pfSense (open-source) at perimeter	Critical
DDoS Protection	Cloudflare Enterprise or AWS Shield Advanced	Critical
VPN Access	WireGuard VPN for all administrative access	Critical
Bastion Host	SSH jump box with MFA, no direct server access	Critical
Network Segmentation	VLAN isolation: DMZ, App, Data, Management subnets	High
Intrusion Detection	Suricata IDS/IPS at network perimeter	High
HSM	Hardware Security Module for encryption keys (optional)	Medium

6.2 Software Security — Authentication and Authorization

Feature	Current Implementation	Enterprise Enhancement
Authentication	JWT tokens (access + refresh)	Add token rotation, short-lived tokens (15min)
Authorization	4-level RBAC (Super, Provincial, Municipal, Ward Admin)	Add attribute-based access control (ABAC)
MFA	Email OTP, SMS OTP, WhatsApp OTP	Add TOTP (Authenticator app), backup codes
Session Management	Redis-backed sessions	Add session fingerprinting, geo-anomaly detection
Password Policy	bcrypt hashing	Enforce: 12+ chars, complexity, breach DB check
Emergency Access	Emergency access system with audit trail	Keep as-is, add time-limited break-glass

6.3 API Security

Measure	Implementation
Rate Limiting	Per-endpoint limits (existing <code>rateLimiting.ts</code>): Auth: 5/min, API: 100/min, Bulk: 10/hour
CORS	Strict origin whitelist (effmemberportal.org only)
Input Validation	Joi schema validation on all endpoints (existing <code>validation.ts</code>)
SQL Injection	Parameterized queries (Prisma ORM + raw SQL with <code>\$1</code> params)
XSS Prevention	Content-Security-Policy headers, React auto-escaping
CSRF	SameSite cookies, CSRF token for state-changing operations
Security Headers	Existing <code>securityMiddleware.ts</code> : HSTS, X-Frame-Options, X-Content-Type
Request Size	Maximum payload: 10MB (configurable)
API Versioning	All routes under <code>/api/v1/</code> prefix

6.4 Database Security

Measure	Implementation
Encryption at Rest	PostgreSQL TDE (Transparent Data Encryption) or LUKS disk encryption
Encryption in Transit	SSL/TLS connections required (<code>sslmode=require</code>)
Access Control	Separate DB users: <code>app_user</code> (limited), <code>admin_user</code> (DDL), <code>readonly_user</code> (replicas)
Connection Pooling	PgBouncer with connection limits per user
Audit Logging	<code>pgaudit</code> extension for DDL/DML logging
Backup Encryption	AES-256 encryption on all backup files
Row-Level Security	Province-scoped data access for provincial admins
Sensitive Data	ID numbers, cell numbers encrypted at column level

6.5 Secrets Management

Item	Current	Enterprise
Environment Vars	<code>.env</code> files	HashiCorp Vault or AWS Secrets Manager

Item	Current	Enterprise
DB Credentials	In <code>.env</code>	Vault dynamic secrets with TTL rotation
JWT Secret	Static in <code>.env</code>	Rotated monthly via Vault
API Keys	WhatsApp, SMS providers in <code>.env</code>	Vault KV with audit trail
SSL Certs	Let's Encrypt on disk	cert-manager in K8s, auto-renewal

6.6 Audit and Compliance

Requirement	Implementation
Audit Logging	Every API mutation logged to <code>audit_logs</code> table (existing <code>auditLogger.ts</code>)
Log Retention	2 years in database, 7 years in cold storage (POPIA compliance)
POPIA Compliance	Data minimization, consent management, right to erasure
Access Logs	Nginx access logs + application audit logs
Change Management	Git-based CI/CD with approval gates
Vulnerability Scanning	Weekly Trivy scans on container images
Penetration Testing	Annual external pen test by accredited firm
Security Training	Quarterly security awareness for admin users

7. Kubernetes Deployment Strategy

See Diagram: "Kubernetes Deployment Architecture" (rendered above)

7.1 Cluster Topology

Component	Count	Spec	Role
Master Nodes	3	8 vCPU, 16GB RAM, 100GB SSD	Control plane (etcd, API server, scheduler)
Worker Nodes	6	16 vCPU, 64GB RAM, 200GB NVMe	Application workloads
Total Cluster	9 nodes	144 vCPU, 432GB RAM	

7.2 Namespace Strategy

Namespace	Purpose	Components
ingress	Traffic entry point	Nginx Ingress Controller, cert-manager
app	Application workloads	Frontend, Backend, WebSocket, Workers
data	Stateful data services	PostgreSQL (Patroni), Redis Sentinel
monitoring	Observability	Prometheus, Grafana, Loki, AlertManager
jobs	Scheduled tasks	CronJobs for birthday SMS, view refresh, backups

7.3 Deployment Specifications

Backend API Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: eff-backend
  namespace: app
spec:
  replicas: 6
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2
      maxUnavailable: 1
  template:
    spec:
      containers:
      - name: backend
        image: eff-registry/backend:latest
        resources:
          requests:
            cpu: "500m"
            memory: "512Mi"
          limits:
            cpu: "2000m"
            memory: "1Gi"
        ports:
        - containerPort: 5000
        readinessProbe:
          httpGet:
            path: /api/v1/health
            port: 5000
          initialDelaySeconds: 10
          periodSeconds: 5
        livenessProbe:
          httpGet:
            path: /api/v1/health
            port: 5000
          initialDelaySeconds: 30
          periodSeconds: 10
```

Horizontal Pod Autoscaler

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: eff-backend-hpa
  namespace: app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: eff-backend
  minReplicas: 6
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 75

```

7.4 Pod Resource Allocation Summary

Deployment	Min Replicas	Max Replicas	CPU Request	Memory Request	CPU Limit	Memory Limit
Frontend	3	10	250m	256Mi	1000m	512Mi
Backend API	6	20	500m	512Mi	2000m	1Gi
WebSocket	3	8	250m	256Mi	1000m	512Mi
Workers	4	12	1000m	1Gi	4000m	2Gi
PostgreSQL	3 (StatefulSet)	3	4000m	16Gi	16000m	64Gi
Redis	3 (StatefulSet)	3	1000m	4Gi	4000m	16Gi

7.5 Service Mesh (Optional — Istio / Linkerd)

For enterprise scale, a service mesh provides:

- **mTLS:** Automatic mutual TLS between all pods
- **Traffic Management:** Canary deployments, circuit breaking

- **Observability:** Distributed tracing (Jaeger), service-to-service metrics
- **Policy:** Fine-grained access control between services

Recommendation: Start without a service mesh. Add Linkerd (lightweight) when the system reaches 50+ pods.

8. Server Procurement Recommendations

8.1 Option A: On-Premises (Colocation in SA Data Center)

Recommended SA Data Centers: Teraco (Johannesburg), Africa Data Centres (Midrand), Dimension Data (Cape Town DR)

Database Servers (3x)

Spec	Recommendation
Server	Dell PowerEdge R750xs or HPE ProLiant DL380 Gen11
CPU	2x Intel Xeon Gold 6430 (32 cores / 64 threads each)
RAM	256GB DDR5 ECC (expandable to 1TB)
Storage	8x 1.92TB NVMe SSD in RAID-10 (7.68TB usable)
RAID	Dell PERC H755N or HPE Smart Array
Network	2x 25GbE SFP28 (bonded)
Power	Dual redundant 1400W PSU
Est. Cost	R450,000 – R550,000 each

Kubernetes Nodes (9x — 3 master + 6 worker)

Spec	Master Nodes (3x)	Worker Nodes (6x)
Server	Dell PowerEdge R650xs	Dell PowerEdge R750xs
CPU	1x Intel Xeon Gold 5418Y (24C/48T)	2x Intel Xeon Gold 6430 (32C/64T each)
RAM	64GB DDR5 ECC	256GB DDR5 ECC
Storage	2x 480GB SSD RAID-1	2x 1.92TB NVMe SSD + 2x 480GB SSD
Network	2x 10GbE	2x 25GbE
Est. Cost	R180,000 – R220,000 each	R380,000 – R450,000 each

Network Equipment

Equipment	Recommendation	Est. Cost
Core Switch	Cisco Catalyst 9300-48T or Arista 7050SX3	R120,000
Firewall	Palo Alto PA-440 or Fortinet FortiGate 200F	R150,000

Equipment	Recommendation	Est. Cost
Load Balancer	F5 BIG-IP i2600 or Nginx Plus (software)	R80,000
UPS	APC Smart-UPS 3000VA (2x per rack)	R60,000

Total On-Premises Capital Expenditure

Item	Qty	Unit Cost	Total
DB Servers	3	R500,000	R1,500,000
K8s Masters	3	R200,000	R600,000
K8s Workers	6	R415,000	R2,490,000
Networking	1 set	R410,000	R410,000
Colo (monthly)	12 months	R45,000	R540,000
Grand Total (Year 1)			R5,540,000
Monthly OpEx (Year 2+)			~R45,000/mo

8.2 Option B: Cloud (Recommended for Agility)

Recommended Providers: AWS (Cape Town Region af-south-1), Azure South Africa North, or Hetzner Cloud (European, cost-effective)

AWS af-south-1 Enterprise Estimate

Service	Spec	Monthly Cost (USD)	Monthly Cost (ZAR)
EKS Cluster	Managed K8s control plane	\$73	R1,350
EC2 Workers	6x m6i.4xlarge (16vCPU, 64GB)	\$4,608	R85,250
RDS PostgreSQL	db.r6g.4xlarge Primary (16vCPU, 128GB)	\$2,500	R46,250
RDS Read Replicas	2x db.r6g.2xlarge (8vCPU, 64GB)	\$2,400	R44,400
ElastiCache Redis	3x cache.r6g.xlarge (26GB)	\$1,350	R24,975
ALB	Application Load Balancer	\$50	R925
S3 Storage	1TB + transfer	\$100	R1,850
CloudWatch	Monitoring + logs	\$200	R3,700

Service	Spec	Monthly Cost (USD)	Monthly Cost (ZAR)
WAF + Shield	DDoS protection	\$150	R2,775
Data Transfer	2TB/month outbound	\$180	R3,330
Monthly Total		\$11,611	~R214,805/mo
Annual Total		\$139,332	~R2,577,660

8.3 Recommendation

Criteria	On-Premises	Cloud (AWS)
Year 1 Cost	R5,540,000	R2,577,660
Year 3 Total Cost	R6,620,000	R7,732,980
Year 5 Total Cost	R7,700,000	R12,888,300
Time to Deploy	8-12 weeks	1-2 weeks
Scaling Speed	Days (hardware procurement)	Minutes (auto-scaling)
Maintenance	In-house team required	AWS managed
Disaster Recovery	Manual setup	Multi-AZ built-in

Recommendation: Start with **Cloud (AWS or Azure)** for Year 1-2 to validate scale, then evaluate migration to **hybrid** (on-premises for DB, cloud for app tier) at Year 3 if cost savings justify the operational overhead.

9. Virtualization Strategy

9.1 Containerization (Docker)

All application components are containerized using multi-stage Docker builds:

```
# Example: Backend Dockerfile (multi-stage)
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npx tsc

FROM node:20-alpine AS runtime
WORKDIR /app
RUN apk add --no-cache chromium # For Puppeteer PDF generation
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 5000
CMD ["node", "dist/app.js"]
```

Container Images:

Image	Base	Size (Est.)	Purpose
eff-backend	node:20-alpine	~250MB	API server
eff-frontend	nginx:alpine	~50MB	Static SPA served by Nginx
eff-worker	node:20-alpine + chromium	~500MB	Background jobs + PDF gen
eff-websocket	node:20-alpine	~200MB	WebSocket server

9.2 Virtualization Layer (if On-Premises)

For on-premises deployments, use VMware vSphere or Proxmox VE:

Option	Recommendation	License Cost
VMware vSphere 8	Enterprise Plus with vSAN	R250,000/year (6-socket)
Proxmox VE	Open-source KVM/LXC	Free (Enterprise support: R30,000/yr)
Recommendation	Proxmox VE for cost savings, VMware if existing enterprise agreement	

VM Allocation on Physical Hosts:

Physical Server	VMs Hosted	Total Allocated
DB Server 1	PostgreSQL Primary (128GB RAM)	1 VM
DB Server 2	PostgreSQL Replica 1 (64GB) + Redis 1 (32GB)	2 VMs
DB Server 3	PostgreSQL Replica 2 (64GB) + Redis 2 (32GB)	2 VMs
K8s Master 1-3	1 K8s master per server	3 VMs
K8s Worker 1-6	1 K8s worker per server (bare-metal preferred)	6 VMs

9.3 Container Registry

Option	Description	Cost
Harbor	Self-hosted, open-source, vulnerability scanning	Free
AWS ECR	Managed, integrated with EKS	~R500/mo
GitHub Container Registry	If using GitHub for source	Free (public), R200/mo (private)

Recommendation: Harbor for on-premises, AWS ECR for cloud deployments.

10. Disaster Recovery and Business Continuity

10.1 RPO/RTO Targets

Tier	RPO (Data Loss)	RTO (Downtime)	Components
Tier 1 (Critical)	0 minutes	15 minutes	PostgreSQL Primary, Redis
Tier 2 (Important)	15 minutes	1 hour	Backend API, WhatsApp Bot
Tier 3 (Standard)	1 hour	4 hours	Frontend, Monitoring, Workers

10.2 Backup Strategy

Data	Method	Frequency	Retention	Storage
PostgreSQL	pgBackRest continuous WAL archiving	Continuous	30 days full, 1 year incremental	Off-site encrypted S3
PostgreSQL	Full dump (pg_dump)	Daily 2AM	30 days	Local + off-site
Redis	RDB snapshots + AOF	Every 60 seconds	7 days	Local
File Uploads	S3 cross-region replication	Real-time	Indefinite	Secondary region
Kubernetes	etcd snapshots	Every 6 hours	14 days	Off-site

10.3 Geographic Redundancy

Site	Location	Role	Components
Primary	Johannesburg (Teraco / AWS af-south-1a)	Active	Full stack
DR	Cape Town (Dimension Data / AWS af-south-1b)	Warm standby	DB replica, reduced app tier
Failover Time			15-30 minutes (automated with DNS failover)

11. Cost Estimates Summary

11.1 Cloud Deployment (Recommended — AWS/Azure)

Category	Monthly (ZAR)	Annual (ZAR)
Compute (K8s + Workers)	R86,600	R1,039,200
Database (PostgreSQL HA)	R90,650	R1,087,800
Cache (Redis Sentinel)	R24,975	R299,700
Networking (LB + Transfer)	R4,255	R51,060
Storage (S3 + Block)	R1,850	R22,200
Monitoring + Security	R6,475	R77,700
Infrastructure Total	R214,805	R2,577,660
WhatsApp Business API	R15,000	R180,000
SMS Provider (bulk)	R50,000	R600,000
SSL Certificates	R0 (Let's Encrypt)	R0
Domain + DNS	R500	R6,000
Services Total	R65,500	R786,000
Grand Total	R280,305	R3,363,660

11.2 Team Requirements

Role	Count	Purpose
DevOps Engineer	1-2	K8s management, CI/CD, monitoring
Backend Developer	2-3	API development, WhatsApp bot, integrations
Frontend Developer	1-2	React UI, dashboard, reports
DBA	1 (part-time)	PostgreSQL tuning, backups, migrations
Security Engineer	1 (part-time)	Security audits, penetration testing
Project Manager	1	Coordination, stakeholder management

12. Implementation Roadmap

Phase 1: Foundation (Weeks 1-4)

- ☐ Containerize all application components (Docker multi-stage builds)
- ☐ Set up CI/CD pipeline (GitHub Actions → Docker build → Registry)
- ☐ Provision cloud infrastructure (AWS EKS or Azure AKS)
- ☐ Deploy PostgreSQL HA cluster (Primary + 2 Replicas)
- ☐ Deploy Redis Sentinel cluster (3 nodes)
- ☐ Configure Nginx Ingress Controller + cert-manager

Phase 2: Migration (Weeks 5-8)

- ☐ Migrate from single Linode VPS to Kubernetes cluster
- ☐ Set up monitoring stack (Prometheus + Grafana + Loki)
- ☐ Configure horizontal pod autoscalers
- ☐ Implement centralized logging
- ☐ Set up automated backup with pgBackRest
- ☐ Configure alerting (PagerDuty / Slack)

Phase 3: Security Hardening (Weeks 9-10)

- ☐ Deploy WAF (Cloudflare or AWS WAF)
- ☐ Implement secrets management (HashiCorp Vault)
- ☐ Set up network policies (K8s NetworkPolicy)
- ☐ Configure database encryption at rest
- ☐ Implement VPN access for administration
- ☐ Conduct initial security audit

Phase 4: Scale Testing (Weeks 11-12)

- ☐ Load test with simulated 1M member dataset
- ☐ Stress test WhatsApp webhook handling (10,000 msg/min)
- ☐ Benchmark bulk upload with 100,000 row files
- ☐ Validate auto-scaling behavior under load
- ☐ Optimize PostgreSQL query performance for 1M rows
- ☐ DR failover drill

Phase 5: Production Cutover (Week 13)

- ☐ DNS cutover from Linode to new infrastructure

- ☐ Monitor for 72 hours with enhanced alerting
 - ☐ Decommission old Linode VPS
 - ☐ Document runbooks for operations team
-

Appendix A: Architecture Diagrams Reference

The following Mermaid diagrams were rendered as part of this document:

1. **High-Level System Architecture** — Shows all tiers: Edge, Application, Cache, Data, Storage, Monitoring
 2. **Solution Architecture (Component View)** — Frontend modules, API middleware pipeline, service layer, background jobs
 3. **Kubernetes Deployment Architecture** — Namespaces, deployments, HPAs, StatefulSets, CronJobs
 4. **Database Architecture** — PostgreSQL cluster, schema overview, materialized views, backup strategy
 5. **Network Architecture** — Subnet isolation, firewall, VPN, bastion host, monitoring
 6. **CI/CD Pipeline** — Git → Lint → Test → Build → Scan → Deploy → Canary → Production
-

Appendix B: Current vs Enterprise Comparison

Aspect	Current (Linode VPS)	Enterprise (K8s Cluster)
Servers	1 VPS	22+ nodes
CPU	4 vCPU	144+ vCPU
RAM	8GB	432+ GB
Storage	160GB SSD	10+ TB NVMe
Database	Single PostgreSQL	HA Cluster (3 nodes)
Cache	Single Redis	Redis Sentinel (3 nodes)
Backend	1 PM2 process	6-20 pods (auto-scaled)
Frontend	1 static serve	3-10 pods (CDN-backed)
Monitoring	Basic logs	Full observability stack
Backup	Manual scripts	Automated continuous archiving
DR	None	Warm standby in Cape Town
Uptime	~99%	99.95% SLA
Deploy	Manual SSH	Automated CI/CD + Canary

Document prepared by the Development Team — February 2026 Classification: CONFIDENTIAL — For internal use only